SatsPath × Arkade Wallet — MVP Technical Integration Plan

Objective: Enable Arkade wallet users to send/receive Bitcoin via human-readable aliases (e.g., `Truja@sexo.ya`) with automatic rail selection (Lightning → On-chain → Ark), client-side VTXO verification, and encrypted identity management.

Timeline: 4-6 weeks | **Approach**: WASM bindings from Rust core to TypeScript wallet

---

Current Architecture Summary

SatsPath (Rust) — Existing Components

Crate	Purpose	Key Exports
`satspath-core`	Types, crypto, resolvers, registry, profile signing	`PaymentMethod`, `SignedPaymentProfile`, `ChainResolver`, `Registry`
`satspath-router`	Rail selection logic	`quote()`, `select_route()`, `RouteQuote`, `FeeEstimate`
`satspath-cli`	CLI: `init`, `register`, `quote`, `pay`, `preview`	—
`satspathd`	HTTP daemon (`/v1/quote`, `/v1/pay`, `/v1/resolve`)	—
`satspath-wasm`	WASM bindings (crypto only today)	`verify_signed_profile`, `canonical_profile_json`, `topic_for_alias`

Arkade SDK (TypeScript) — Existing Components

- **Tier 1**: VTXO DAG reconstruction + Schnorr/MuSig2 validation
- **Tier 2**: Taproot tree audit + CSV delays + Boltz HTLC support
- **Tier 3**: Sovereign exit — local encrypted storage + ASP-independent broadcast
- **Interfaces**: `IndexerProvider`, `OnchainProvider`, `StorageProvider`

Integration Strategy: WASM Bindings

Extend `satspath-wasm` to expose **resolver chain + router** to TypeScript. Arkade wallet calls:

```
const quote = await satspath.quote("Truja@sexo.ya", 21000n);
// → { selected_method, reason, fee_sats, eta, qr, execution_mode }
```

Why WASM: Single binary, works in browser/PWA/React Native, no sidecar process.

MVP Scope (4-6 Weeks)

Phase 1: Extend satspath-wasm (Week 1-2)

Goal: Expose resolver + router to TypeScript via WASM

Tasks: 1. Add `wasm` feature flags to `satspath-core` and `satspath-router` (remove `tokio`/`reqwest` dependencies) 2. Create WASM-compatible resolver implementations using `web-sys` fetch 3. Export `quote()`, `verify_signed_profile()`, `ChainResolver`, `Registry` from `satspath-wasm` 4. Add `fingerprint_pubkey()`, `mask_identifier()` helpers

Files to modify: - `crates/satspath-core/Cargo.toml` — add `wasm` feature - `crates/satspath-router/Cargo.toml` — add `wasm` feature - `crates/satspath-wasm/src/lib.rs` — new exports - `crates/satspath-wasm/src/resolver.rs` — NEW: WASM resolver chain - `crates/satspath-wasm/src/router.rs` — NEW: router wrapper

Phase 2: Build WASM Package (Week 2)

Goal: Publish `@satspath/wasm` npm package

Tasks: 1. Configure `wasm-pack` build for web target 2. Generate TypeScript definitions via `wasm-bindgen` 2. Publish to local npm registry or GitHub Packages 3. Verify import in TypeScript: `import { quote } from '@satspath/wasm'`

Phase 3: Arkade Wallet Service Layer (Week 2-3)

Goal: Integrate WASM into Arkade wallet services

New file: `arkade-wallet/src/services/satspath.ts`

```
import { quote, verify_signed_profile, ChainResolver } from '@satspath/wasm';

export async function getPaymentQuote(alias: string, amountSats: bigint): Promise<PaymentQuote>
export function verifyProfile(profile: SignedPaymentProfile): boolean
export async function resolveProfile(alias: string): Promise<SignedPaymentProfile>
```

Types: `arkade-wallet/src/types/satspath.ts` (mirror Rust types)

Phase 4: Send/Receive UI Flows (Week 3-4)

Goal: User-facing flows in Arkade wallet

Send Flow (`SendFlow.tsx`):

```
Input: alias + amount → satspath.quote() → shows rail + fee + QR → Pay button
```

Receive Flow (`ReceiveFlow.tsx`):

```
Manage identity keypair → Add payment methods → Sign profile → Publish (HTTPS/Nostr)
```

Components: - `RailSelector.tsx` — Shows selected rail, reason, fee, ETA - `OwnershipBadges.tsx` — / per method verification status - `ArkVtxoStatus.tsx` — VTXO verification progress + sovereign exit status

Phase 5: Ark VTXO Verification (Week 4)

Goal: When Ark rail selected, run Arkade SDK verification

```
// Ark Receive
async function onArkReceive(vtxoOutpoint: Outpoint, arkMethod: ArkMethod) {
  const report = await verifyVtxoChain(vtxoOutpoint, indexer, onchain);
  if (report.valid) {
    await storage.setItem(`ark_exit:${vtxoOutpoint}`, encrypt(report.exitData));
    showUI("VTXO verified - sovereign exit ready");
  }
}

// Ark Send
async function sendViaArk(quote: RouteQuote, amountSats: bigint) {
  const intent = await arkade.createTransferIntent({
    server: arkMethod.server,
    recipientPubkey: arkMethod.pubkey,
    amountSats,
```

```
  });
}
```

Phase 6: Profile Management + Ownership Proofs (Week 4-5)

Goal: Full identity lifecycle

- Generate secp256k1 identity keypair (encrypted at rest via Web Crypto API)
- Per-method ownership proofs:
  - **Lightning**: Sign LNURL-pay callback challenge
  - **On-chain**: Sign message with address key
  - **Ark**: `ArkOwnershipProof` from ASP
- Profile publication: HTTPS well-known → Nostr NIP-05

Phase 7: Testing + E2E (Week 5-6)

- Testnet Lightning payment (LDK)
- Testnet On-chain payment (BDK)
- Testnet Ark payment + VTXO verification
- Fee estimation accuracy
- Error handling / edge cases

Technical Details

WASM Feature Flags

```toml
# satspath-core/Cargo.toml
[features]
default = ["std"]
wasm = []  # No tokio, reqwest, tokio-tungstenite

# satspath-router/Cargo.toml
[features]
default = ["std"]
wasm = ["satspath-core/wasm", "satspath-router/fees-wasm"]

# satspath-wasm/Cargo.toml
[dependencies]
satspath-core = { path = "../satspath-core", features = ["wasm"] }
satspath-router = { path = "../satspath-router", features = ["wasm"] }
wasm-bindgen = "0.2"
wasm-bindgen-futures = "0.4"
web-sys = { version = "0.3", features = ["Fetch", "Request", "Response", "Headers", "Window", "Crypto"]
```

WASM Resolver Chain (browser-compatible)

```rust
// satspath-wasm/src/resolver.rs
use web_sys::{Request, RequestInit, RequestMode, Response};
use wasm_bindgen_futures::JsFuture;

pub struct WasmChainResolver {
    local_registry: WasmRegistry,
    bip353_resolver: WasmBip353Resolver,
    https_resolver: WasmHttpsResolver,
```

```
    nostr_resolver: WasmNostrResolver,
}

impl WasmChainResolver {
    pub async fn resolve_alias(&self, alias: &str) -> Result<SignedPaymentProfile, JsValue> {
        // Try each resolver in order: local → BIP353 → HTTPS → Nostr
    }
}
```

TypeScript Types (auto-generated + manual)

```
// arkade-wallet/src/types/satspath.ts
export type PaymentMethod =
  | { type: "Lightning"; label: string; lightning_address?: string; lnurl?: string; bolt12?: string; re
  | { type: "Onchain"; label: string; network: "mainnet" | "testnet" | "regtest"; address?: string; sil
  | { type: "Ark"; label: string; server: string; pubkey: string; vtxo_pointer?: string; opaque_uri?: s

export interface PaymentProfile {
  alias: string;
  identity_pubkey: string;
  methods: PaymentMethod[];
  updated_at: number;
  expires_at?: number;
  sequence?: number;
  preferences: string[];
  nonce?: string;
  method_verifications: MethodVerification[];
}

export interface SignedPaymentProfile {
  profile: PaymentProfile;
  signature: string; // hex Schnorr
}

export interface RouteQuote {
  selected_method: PaymentMethod;
  reason: string;
  estimated_fee_sats: number;
  estimated_confirmation: string;
  fee_snapshot?: FeeSnapshot;
  execution_mode: "Preview" | "MainnetPreview" | "TestnetExperimental" | "ManualWallet";
  wallet_hint: string;
}

export interface PaymentQuote {
  rail: "Lightning" | "Onchain" | "Ark";
  rail_label: string;
  fee_sats: number;
  eta: string;
  reason: string;
  qr_payload: string; // BOLT11 / BIP21 / ark: URI
  ownership_verified: boolean;
  execution_mode: RouteQuote["execution_mode"];
}
```

File Structure (Post-Integration)

```
satspath/
  crates/
     satspath-core/
        Cargo.toml          # +wasm feature
     satspath-router/
        Cargo.toml          # +wasm feature
     satspath-wasm/
         Cargo.toml
         src/
            lib.rs          # exports
            crypto.rs       # existing
            resolver.rs     # NEW: WASM resolver chain
            router.rs       # NEW: router wrapper
            helpers.rs      # NEW: fingerprint, mask
         wasm-pack build --target web
  docs/
     TECHNICAL_PLAN_MVP.md   # this file
  sdk/wasm/pkg/              # generated npm package

arkade-wallet/
  src/
     services/
        satspath.ts         # NEW: SatsPath integration
        arkade.ts           # existing Arkade SDK wrapper
        lightning.ts        # LDK wrapper
        onchain.ts          # BDK wrapper
     components/
        SendFlow.tsx
        ReceiveFlow.tsx
        RailSelector.tsx
        OwnershipBadges.tsx
        ArkVtxoStatus.tsx
     types/
         satspath.ts         # NEW: TS types mirror
  package.json               # + @satspath/wasm dep
```

Risk Mitigation

Risk	Mitigation
WASM bundle too large	`wasm-opt -Oz`, tree-shake unused crate code, lazy-load
Resolver chain fails in browser (CORS/DNS)	Fallback: HTTP daemon proxy for DNS/Nostr; HTTPS well-known works natively
Ark VTXO verification slow	Run async with progress UI; cache verified VTXOs
Profile publication complex	MVP: HTTPS well-known only; Nostr/P2P post-MVP
Fee estimation inaccurate	Use mempool.space API + 10% buffer (already in router)

Success Criteria (MVP Done)

- ☐ User enters `alias@domain` + amount → sees rail + fee + QR in < 2s
- ☐ Lightning payment works via LDK (testnet)
- ☐ On-chain payment works via BDK (testnet)
- ☐ Ark payment shows VTXO verification status (testnet)
- ☐ Profile creation + signing + HTTPS publish works
- ☐ Ownership badges show / per method
- ☐ All crypto in WASM (no native deps)

Next Steps

1. **Approve plan** → proceed to implementation
2. **Phase 1**: Add `wasm` feature flags to `satspath-core` and `satspath-router`
3. **Phase 1**: Implement `WasmChainResolver` in `satspath-wasm`
4. **Phase 1**: Export `quote()` and router types from `satspath-wasm`
5. **Phase 2**: `wasm-pack build` → verify TypeScript imports
6. **Phase 3**: Build Arkade wallet `satspath.ts` service
7. **Phase 4**: Wire UI components

Generated as part of SatsPath × Arkade MVP integration. See `satspath/README.md` for protocol overview.